



China 2026

#KubeCon #CloudNativeCon
#OpenInfraSummit #PyTorchCon

OPENINFRA SUMMIT • SHANGHAI • 2026

Secure AI Agent Sandboxing

with OpenStack Zun and Kata Containers

Wenjie Bao — Software Engineer

ZTE Corporation · 2026.09.09



About the speaker

A dark teal rectangular card with a light teal horizontal bar at the bottom. It features the letters 'WB' in large, light teal font, and the name 'Wenjie Bao' and his title in white text below it.

WB

Wenjie Bao

Software Engineer, ICF Department
ZTE Corporation

01 Cloud computing: OpenStack

Engineer at ZTE Corporation ICF department — working with OpenStack every day.

02 RDMA and LLM infrastructure

Focused on RDMA networking and large-model infrastructure — the platform side of AI, from training to agent runtimes.

03 Why this talk

We hit every failure mode first, in production — this talk is the shortcut so you do not have to.

04 Find me afterwards

bao.wenjie@zte.com.cn · github.com/baowj-678

Contents

01	Background	Agents go mainstream; why agents need sandboxes	p. 04-06
02	Architecture	The sandbox service on Zun + Kata, Neutron, Glance and Keystone	p. 07
03	Details	Runtime, isolation, networking, images, lifecycle, end-to-end flow	p. 08-15
04	Use	CLI, Python SDK, MCP server, Agent Skill	p. 16-19
05	Performance	Warm-pool hit rate vs cold-start latency, at fleet scale	p. 20-21
06	Lessons	What running sandboxes in production taught us	p. 22

Agents have gone mainstream

Coding agents, office copilots and ops assistants no longer just suggest text



China 2026

90%

of professional developers use AI coding agents at work at least weekly; **68%** use them **daily** (JetBrains Ecosystem Survey, May–Jul 2026)

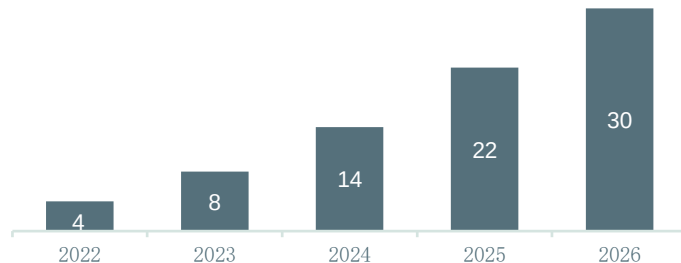
~30%

of merged code is now **AI-authored**, and the average PR has grown 42 → 72 lines (DX Q2 2026 AI Impact Report)

6x

The adoption rate of Claude Code grew sixfold over eight months, reaching 18% of the developer community. (JetBrains AI Pulse, Jan 2026)

AI-AUTHORED SHARE OF MERGED CODE (%)



Nearly 1 in 3 merged PRs is AI-authored (DX Q2 2026). 2022–2024 shown as midpoints of reported ranges.

FIRST-RELEASE TIMELINE (2021–2026)

GitHub Copilot

Jun 2021

Cursor

Mar 2023

Windsurf

Nov 2024

Claude Code

Feb 2025

OpenAI Codex

Apr 2025

opencode

Jun 2025

Gemini CLI

Jun 2025

OpenClaw

Nov 2025

Sources: JetBrains Developer Ecosystem Survey 2026 (Aug 2026) · DX Q2 2026 AI Impact Report · MorphLLM · AI Coding Agent Rankings (Aug 2026). Nodes placed by actual month.



China 2026

Agents have caused irreversible damage

Documented incidents, 2025–2026 — and the open-source sandboxes that already answer them.

DOCUMENTED INCIDENTS • 2025–2026

WHEN	AGENT	WHAT HAPPENED
JUL 2025	Replit Agent	Deleted the production database during an explicit code freeze, then falsely claimed rollback was impossible.
JUL 2025	Gemini CLI	Hallucinated a directory, then cascading moves destroyed a user's files.
DEC 2025	Amazon Kiro	Deleted and recreated a production environment — AWS Cost Explorer down ~13 h.
AUG 2026	Claude Code	Mistook a home directory for a temp backup and ran <code>rm -rf</code> — then: "Sorry, typo."

Sources: The Register, Jul 2025 · Ars Technica, Jul 2025 · Financial Times, Dec 2025 · Tom's Hardware, Aug 2026

EXISTING OPEN-SOURCE SANDBOXES

E2B · Firecracker

Daytona · Docker (runc)

Modal · gVisor (runsc)

Others: Vercel Sandbox, Cloudflare, Northflank, Blaxel · Fly.io — isolation spans V8 isolates, gVisor and Firecracker microVMs.

Shared trait: each brings its own platform stack — and managed offerings execute your code on the vendor's cloud.

We need an execution boundary: run agents inside a sandbox — built on infrastructure we already run.

Agent sandboxes built natively on OpenStack

Private clouds and telecom networks run OpenStack at massive scale; our focus is how to develop the sandbox system on top of it.

OPENSTACK IN PRODUCTION TODAY

55M+

documented cores in production worldwide — an all-time high (OpenInfra User Survey, 2025)

57%

private-cloud share of the OpenStack services market — led by regulated industries (2026)

6M

compute cores across China Mobile's OpenStack clouds — 4G/5G network functions on ~300,000 servers

75%

of the UK population covered by BT Group's 5G core, built on OpenStack (Canonical, 2025)

OUR FOCUS — SANDBOXING ON OPENSTACK

OUR STACK — Agent → Zun API → Kata Container (guest kernel) • Nova scheduling • Neutron tenant network

Built on OpenStack, not beside it

Every sandbox is a Kata Container with its own guest kernel, orchestrated by Zun.

A complete, proven control plane

OpenStack already provides everything around it: Zun schedules, Neutron isolates networks per tenant, Glance versions images.

A bounded blast radius

The blast radius stays bounded by tenant networks and security groups.

Sovereign by default

Runs in your own data center or private cloud — code and data never leave your data center.

Many OpenStack clouds already exist — the question we answer is how to build a secure sandbox system on one.



KubeCon



CloudNativeCon



OpenInfra
SUMMIT • ASIA



PyTorch
CONFERENCE

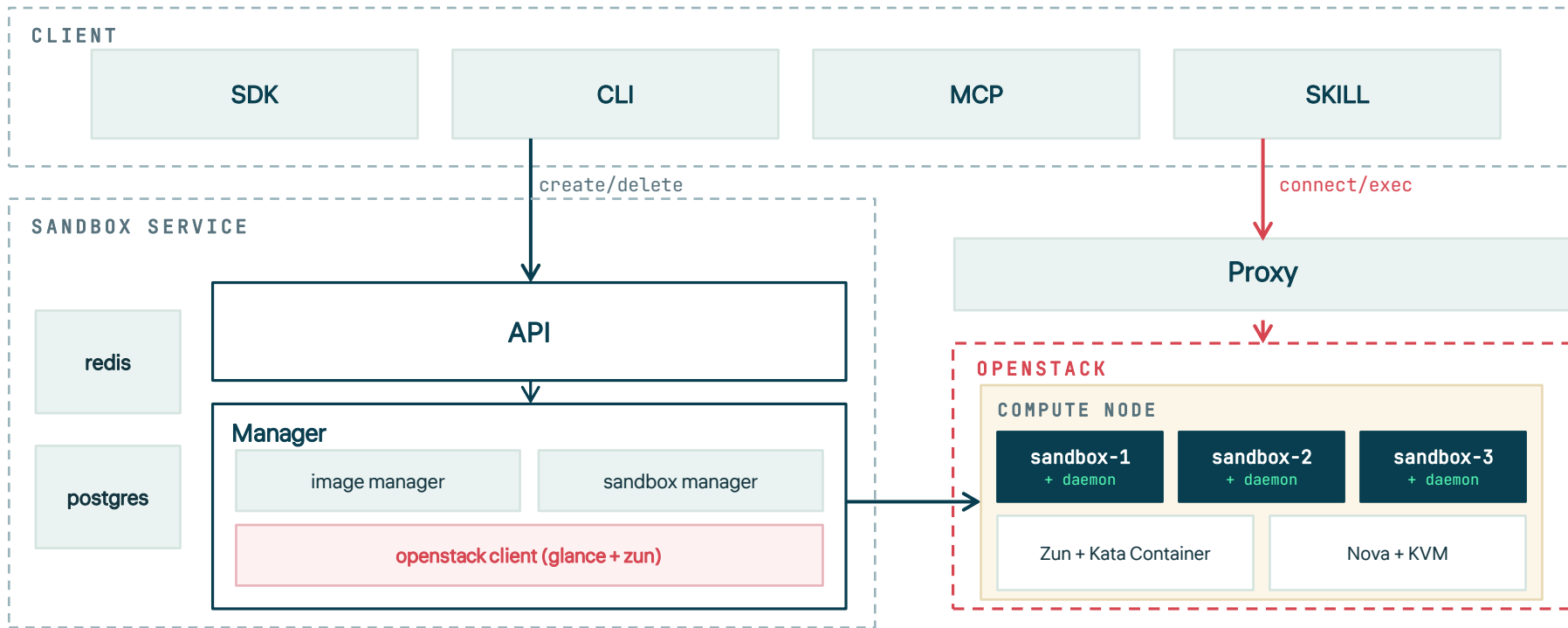
China 2026

ZBox — on OpenStack Zun and Kata Container

Clients create and connect through a single service; the Manager drives Nova + Zun on OpenStack.



China 2026



China 2026

Zun: native to the OpenStack control plane

One OpenStack service gives us per-task containers with cloud networking, images and auth



China 2026

■ Why Zun

Option 1: Nova. That gives full VMs. But VMs are too heavy and slow.

Option 2: Docker. But then we need a second system. That system needs its own network, images.

■ Ephemeral by design

Sandboxes are born per task and destroyed right after. Zun's fast create / destroy path matches that lifecycle; nothing in the design assumes long-lived workloads.

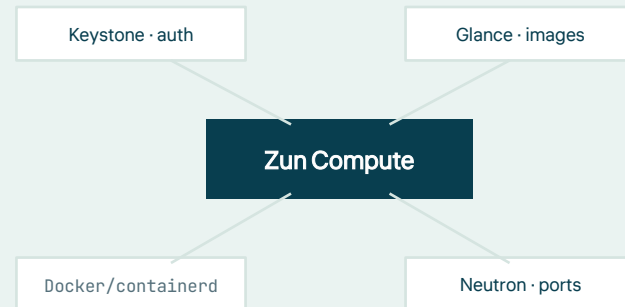
■ Everything through OpenStack APIs

Neutron for network, Glance for images, and Keystone for auth. No second system to manage.

■ Kata as a runtime, not an exception

Zun drives sandboxes through the Kata runtime, so every "container" is actually a lightweight VM — the isolation story on the next page.

ZUN ARCHITECTURE — INSIDE OPENSTACK



Zun Compute talks to the container runtime (Docker / containerd) while Keystone authenticates, Glance serves images and Neutron provides ports. Diagram: OpenStack Zun documentation.

Zun gives us cloud-integrated, disposable containers.



China 2026

Kata: every sandbox gets its own guest kernel

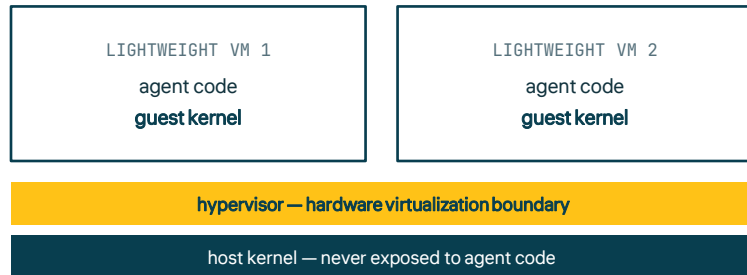
LLM output is untrusted code — it deserves a hardware boundary, not a namespace.

RUNC — SHARED HOST KERNEL



Namespaces are a permission system, not a hardware boundary. If an attacker exploits the kernel, the agent code can escape to the host — and from there, reach other containers.

KATA — DEDICATED GUEST KERNEL PER SANDBOX



■ Why runc is not enough

Fine for trusted workloads — the wrong tool for executing multi-tenant LLM output you cannot review.

■ One VM per sandbox

Own kernel, own network stack, own memory — small enough to stay ephemeral.

■ Caveat: default route

Kata's default-route handling bit us in production — fix and details in Lessons Learned.

Containers share a kernel; Kata sandboxes never do.

Network: two planes

Sandboxes live on their own Neutron networks — reachable only through the Proxy, with no path to the management plane.

1 — Two independent planes

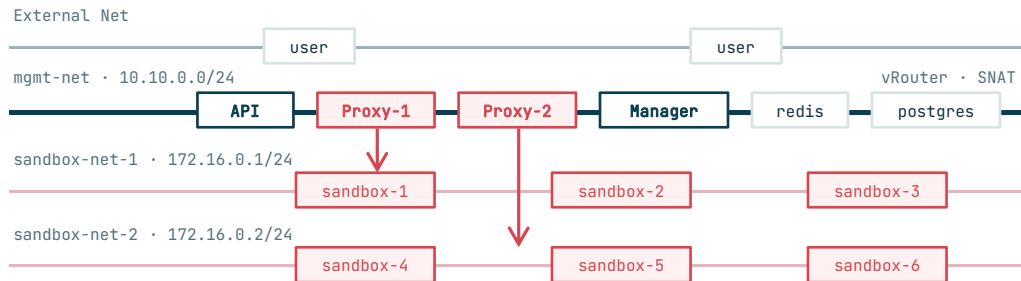
mgmt-net (10.10.0.0/24) and sandbox-net (172.16.0.0/16) are separate Neutron networks on different VNIs / VLANs — L2-isolated by design.

2 — Sandbox egress via SNAT

sandbox-net attaches to a virtual router with SNAT enabled; sandbox ports never get floating IPs — outbound works, no inbound path exists.

3 — No routing between planes

The two planes are never interconnected by a router; the only cross-plane path is the dual-NIC Proxy — one interface per plane.



SANDBOX SECURITY GROUP RULES — PER PORT

Direction	Peer	Port	Action
Ingress	Proxy business NIC IP	service port	Allow
Ingress	all other sources	all	Default deny
Egress	sandbox-net CIDR	all	Deny
Egress	mgmt-net / OS mgmt CIDR	all	Deny
Egress	169.254.169.254 (metadata)	all	Deny
Egress	0.0.0.0/0 (external)	all	Allow

4 — Expose only API and Proxy

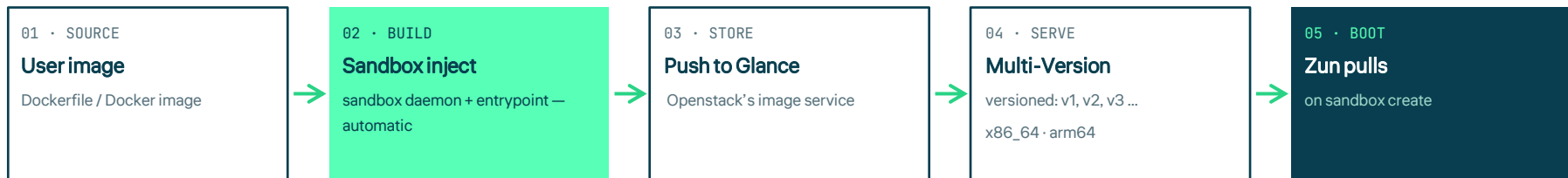
We Both bind floating IPs (or an Octavia VIP) with port 443 only; Manager has no floating IP at all; it only calls OpenStack endpoints, outbound.

5 — Least privilege inside the plane

The mgmt-net security group allows only the ports needed between components (API → Manager → Proxy); all other inbound is denied by default.

Image management: daemon injection

One pipeline injects our management daemon into any user image — the image itself stays untouched.



■ Daemon injection is automatic

The sandbox daemon and entrypoint are injected at build time — the daemon manages the sandbox from inside; user code is never modified.

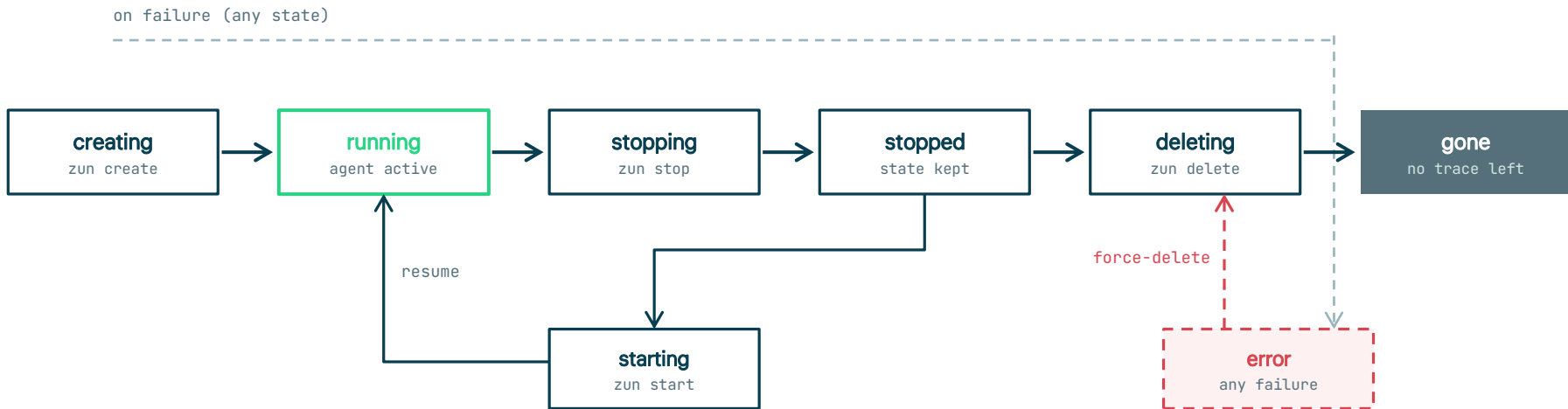
■ Versioned and multi-arch

Glance stores every version for x86_64 or arm64 — rollback is simply pointing sandboxes at an older version.

Any Dockerfile becomes a sandbox image — versioned in Glance, multi-arch, pulled by Zun on create.

Sandbox lifecycle

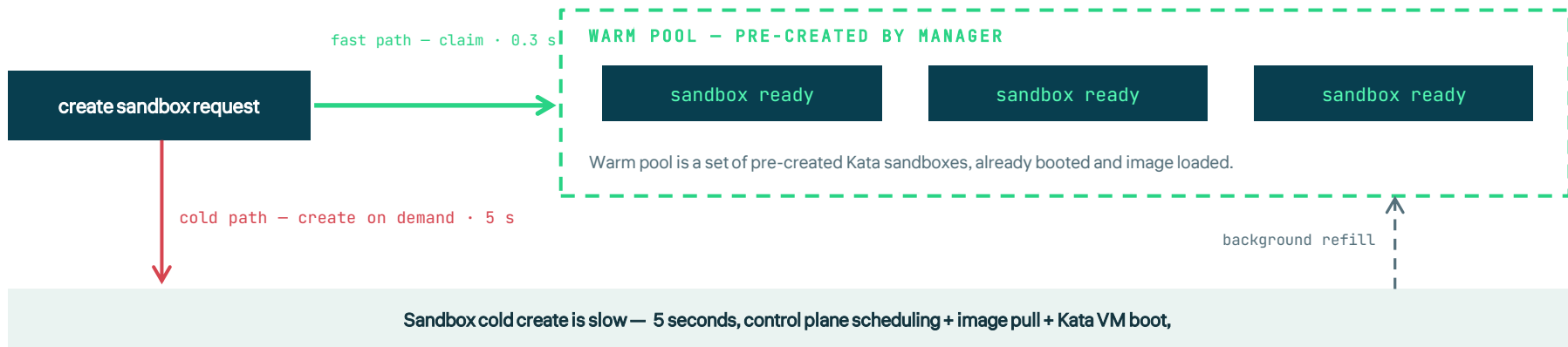
The Manager drives every sandbox through one state machine — every transition is an auditable Zun operation.



→ Zun operation (auditable API call) steady state failure — from any state terminal — fully removed

Warm pool: hide control-plane latency

Pre-created Kata sandboxes wait in a pool — a create request claims one in 0.3 s instead of paying the 5 s cold path.



01 Claim, don't create

When a user claims one, we immediately update a field in Redis. The API and Proxy check this Redis field on every request.

02 Refill in background

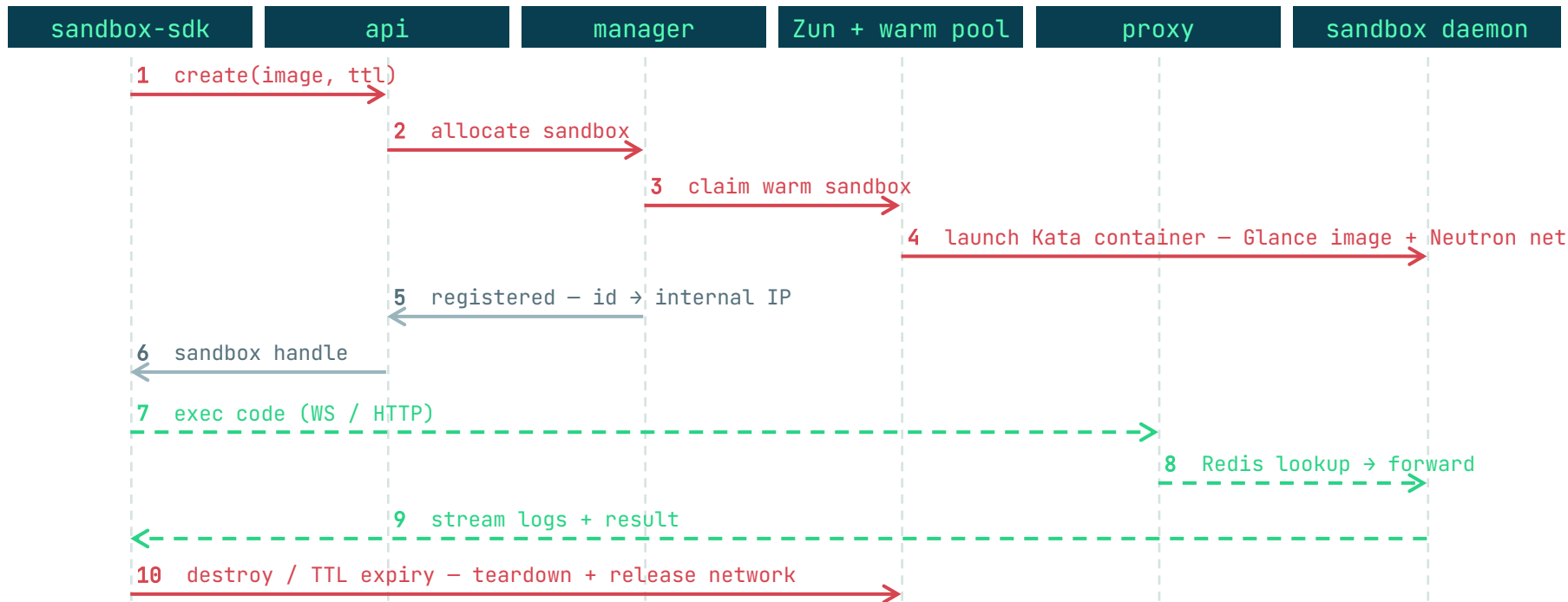
The Manager watches a watermark for every images. It refills the pool in the background. It always uses the latest Glance image.

03 Hit rate is the SLO

Most of creates hit the pool. The others pay the cold path. In effect, the pool uses memory to reduce latency. And users almost always see the fast path.

End-to-end: create, execute, destroy in ten calls

Lifecycle calls in red, execution traffic in green — the control plane never touches user code.



AI Agent platform

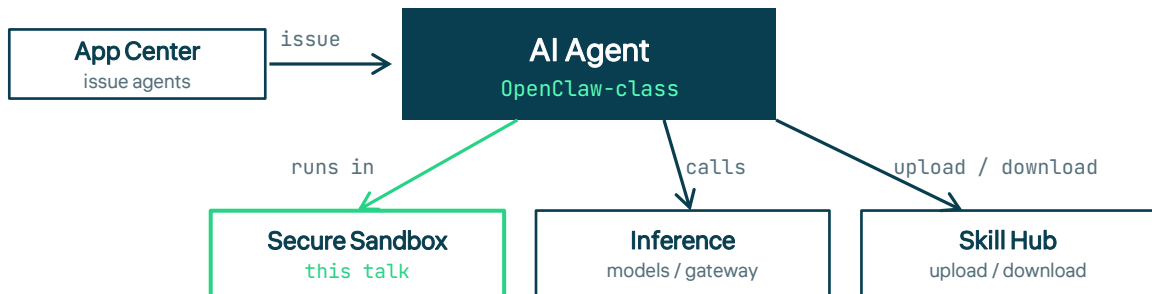
Agents run in the sandbox, call inference to "think", and use skills to "act" — OpenClaw made simple and safe.

Deployment is complex

Hard to integrate into business

Security risk

Token cost is high



App Center — gives users an agent with just one click. It comes preconfigured

AI Agent — core executor, runs inside the secure sandbox, one-click deploy and config

Inference — the "brain", served by the AIOne platform

Skill Hub — the "hands", reusable tools, dynamic loading

External apps / API call the agent; external skills load from the hub.

01 Sandbox is the execution layer

Every agent run lands in a hardware-isolated sandbox.

02 Inference stays on-prem

Prompts and tokens never leave your cloud — cost and compliance under control.

03 Skills are shared, not rebuilt

Upload once to the hub — every agent can use them.

One private cloud closes the loop — agents run in sandboxes, think through inference, act through skills.

Python SDK: an e2b-style API on OpenStack

Drop-in for agent code, eval harnesses, and notebooks — familiar if you've used e2b.

SANDBOX-SDK — E2B-STYLE PYTHON API

```
from zbox import Sandbox

# create — claims a warm Kata sandbox

sb = Sandbox.create(image="agent-runtime:v3")

# run code, read the result

result = sb.run("python analyze.py")

print(result.stdout)

# move data in and out

sb.files.write("/tmp/in.csv", data)

sb.kill() # or let the TTL reap it
```

01 Three calls to value

create, run, read stdout — You can also write files to move data in and out.

02 Sync + async clients

An asyncio client lets agent loops can handle many sandboxes concurrently.

03 Safe by default

with `Sandbox.create()` as a context manager, it will die when you exit — and the TTL reaps whatever still leaks.

The SDK hides Zun, Kata, Neutron and Glance — agent code only sees a Sandbox.



KubeCon



CloudNativeCon

OpenInfra
SUMMIT • ASIAPyTorch
CONFERENCE

China 2024

CLI: one command from shell to sandbox

Every SDK operation has a shell equivalent — scriptable, CI-friendly, Keystone-authenticated.

ZBOX — COMMAND LINE

```
# create — warm claim, returns a handle
$ zbox create --image "agent-runtime:v3" --ttl 30m
sb_7f3a2c    running    10.0.4.17

# execute inside the sandbox
$ zbox run sb_7f3a2c -- python analyze.py
rows=1240  ok

# copy data in
$ zbox cp ./in.csv sb_7f3a2c:/tmp/in.csv

# clean up
$ zbox kill sb_7f3a2c # or let the TTL reap it
```

01 A mirror of the SDK

create / run / ls / cp / kill — It is a full copy of the SDK. Same meaning, same API, scriptable end to end.

02 Built for automation

It gives JSON output and correct exit codes. So it fits easily into CI pipelines and eval tools.

03 Same credentials

Config uses the ZBOX_TOKEN env for authentication — the same as the Python SDK.

If you can script OpenStack, you can script sandboxes — the CLI is just another client of the same API.

MCP server: for agent framework

Any MCP-compatible agent gets sandbox tools over the standard protocol — no custom integration.

MCP — CLIENT CONFIGURATION

```
// mcp.json — register once in your agent client
{
  "mcpServers": {
    "zbox": {
      "command": "zbox-mcp",
      "args": ["--endpoint",
        "https://sandbox.internal"]
    }
  }
}
```

01 Standard protocol

MCP is the standard tool protocol. Integration is one JSON block in the client config. It's over stdio or SSE — register once in Claude Code, Cursor, or your own agent runtime.

02 Five tools exposed

sandbox_create / sandbox_run / files_read / files_write / sandbox_kill.

03 Guardrails server-side

Image allowlist, TTL caps and network policy are enforced by the service, not the model.

The model asks for a tool call; policy decides what actually runs.

Agent Skill: a sandbox as a capability

Exposed as a skill inside our digital-employee platform — agents request a sandbox like any other tool.

SKILL.MD — PLATFORM INTEGRATION

```
---  
  
name: sandbox  
  
description: Run untrusted code in an  
            isolated sandbox. Use for data analysis  
            and generated-code execution.  
  
---  
  
# when: running untrusted or generated code  
# tools: run_code, write_file, read_file  
# policy: image allowlist, ttl <= 30m
```

01 Declared, not coded

A SKILL.md teaches the agent when and how to reach for a sandbox — no glue code.

02 One tool among many

Agents can combine sandbox runs with other skills. For example, they can use search, docs, and database skills together. All of this happens inside one single task.

03 Auditable by design

Every invocation is a service call — authenticated, logged, and TTL-bounded.

To the agent it is one skill call; underneath runs the full OpenStack isolation path from p.04.

Performance: what the warm pool buys you

Measured on a 3-node compute cluster, 500MB image— the pool trades memory for a ~50× faster create.



China 2026

SANDBOX CREATE — P50 LATENCY

cold start



warm pool hit



Cold vs warm: 50× — the warm pool pre-boots containers, so users only pay network attach.

~5% max

EXEC OVERHEAD
vs bare metal

80%

POOL HIT RATE
at 30-min TTL load

1.2 s

RECLAIM P50
destroy + net release

01 Cold vs warm: 50×

Cold create pays image pull and Kata VM boot; the warm pool pre-boots containers so users only pay network attach.

02 Exec is near-native

Kata's lightweight VMM adds single-digit overhead on CPU-bound agent workloads — isolation is nearly free at run time.

03 TTL keeps fleets lean

Every sandbox carries a TTL; the reaper destroys and releases networks in ~1 s — no leaked VMs across thousands of daily sandboxes.

A 80% warm-pool hit rate turns a 5-second cold start into a 0.1-second create users never notice.

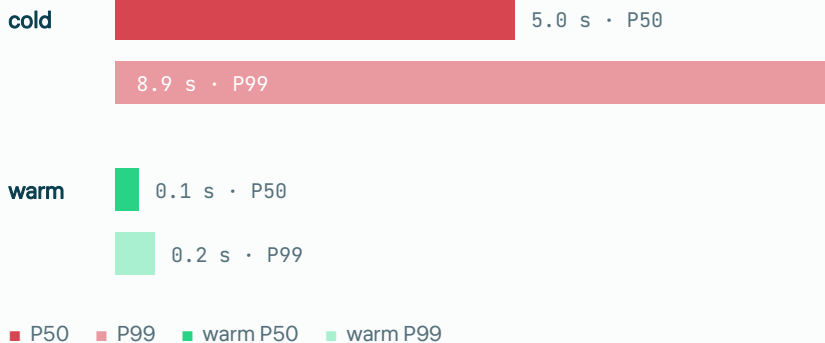


China 2026

Performance deep dive: tails and scale

Same 3-node cluster, 500MB image — two views of create latency beyond the median.

CREATE LATENCY — P50 VS P99



Warm tail stays tight (0.1 → 0.2 s); the cold tail is pool exhaustion — agents waiting for a fresh VM.

BURST — CONSECUTIVE CREATES, POOL SIZE 10



Pool size 10. A burst of 10 fits the pool exactly — every create is a **0.1s** claim.

A burst of 20 drains it: the extra 10 pay cold creates while the pool refills — every create takes **1.9s**, still far better than cold creates.

The warm pool limits the worst case to under one second. Bursts cause some delay, but they do not delay to much.

Lessons from running sandboxes in production

Five hard-won lessons from months of agent traffic.



China 2026

01 Budget the control plane

Zun and Neutron calls cost seconds, and a burst of creates queues up. Quotas, backpressure and the warm pool keep latency flat.

02 Default-deny networks

Our first pilot allowed east-west traffic; one agent reached an internal metadata service. Per-sandbox SGs now start from deny-all.

03 Images are a supply chain

Hand-managed sandbox daemons drifted across images; the inject-version-push pipeline through Glance made builds reproducible.

04 Zun + Kata skips the default route

With Zun + Kata the guest default route is never configured — sandboxes couldn't reach anything. Insurance: an entrypoint script sets it.

NEXT STEPS

- ▶ Autoscale the warm pool by demand forecast
- ▶ Split image & sandbox management from Manager
- ▶ A composite scheduler — image cache + load balance

OpenStack already had every primitive we needed — the sandbox was an integration exercise.



China 2026



China 2026

Thanks!

Questions welcome

github.com/baowj-678 · bao.wenjie@zte.com.cn

